
EEEM071 Advanced Topics in Computer Vision and Deep Learning

Lab Week 9

Pytorch Optimization

Dr. Xiatian Zhu
xiatian.zhu@surrey.ac.uk

1 Introduction

In this lab, we focus on three important components (*i.e.*, criterion (or loss function), optimizer, and scheduler) for loss optimization as shown in Fig. 1 in an image classification task (open this [Colab notebook](#)).

```
label_smoothing = 0.0
criterion = nn.CrossEntropyLoss(label_smoothing=label_smoothing)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[5], gamma=0.1)
```

Figure 1: Three important components that can affect the final performance of a task.

2 Cross Entropy

Cross-entropy is a commonly used loss function in classification tasks in machine learning. In classification, the goal is to assign an input to one of several predefined categories, based on its features or attributes.

Given an input instance and a corresponding true label or target, the cross-entropy loss function measures the difference between the predicted class probabilities and the true class probabilities. The predicted class probabilities are output by the classification model, and represent the model's belief or confidence in each of the possible classes for the input instance.

Cross-entropy is defined as follows: $H(p, q) = -\sum_x p(x) \log(q(x))$ where p is the true probability distribution, q is the predicted probability distribution, and x is a possible outcome.

The cross-entropy loss function penalizes the model when the predicted class probabilities deviate significantly from the true class probabilities. This penalty encourages the model to learn to accurately predict the true class probabilities for each input instance, and to assign higher probabilities to the true class compared to the other classes.

You can define a cross-entropy loss in PyTorch by simply typing the following code in Fig. 2.

```
import torch
import torch.nn as nn

loss_function = nn.NLLLoss()
pred = torch.tensor([[0.1, 0.5, 0.4], [0.5, 0.4, 0.1], [0.2, 0.2, 0.6]])
target = torch.tensor([0, 1, 2])
loss = loss_function(torch.log(pred), target)
print('Average loss', loss)

# without reduction -- average
loss_function = nn.NLLLoss(reduction='none')
pred = torch.tensor([[0.1, 0.5, 0.4], [0.5, 0.4, 0.1], [0.2, 0.2, 0.6]])
target = torch.tensor([0, 1, 2])
loss = loss_function(torch.log(pred), target)
print('Loss for each sample', loss)

# when the output is not a "softmax" output, we usually call it logits
loss_function = nn.CrossEntropyLoss()
logits = torch.tensor([[0.5, 3.2, -0.2], [-2, 3, 0.1], [-4, 1.2, 1]])
target = torch.tensor([0, 1, 2])
loss = loss_function(logits, target)
print('Average loss', loss)
```

Average loss tensor(1.2432)
Loss for each sample tensor([2.3026, 0.9163, 0.5108])
Average loss tensor(1.2190)

Figure 2: Assume `pred` is a softmax prediction (summation is 1) from your model, `nn.NLLLoss` (negative log-likelihood, which is also another name for cross-entropy) is used as the loss function. Note that the input to this `NLLLoss` required a log version of the softmax prediction. When the prediction is real value output, we usually call it `logits`. The corresponding loss function then becomes `nn.CrossEntropyLoss`. Note that the difference is that `logits` will be normalized through a `LogSoftmax` function implicitly, then will still pass to `nn.NLLLoss` for loss computation.

2.1 Label Smoothing

Label smoothing is a regularization technique used in machine learning to prevent overfitting and improve model generalization. It involves replacing the one-hot encoding of the true labels with a smoothed distribution that assigns some probability mass to incorrect labels.

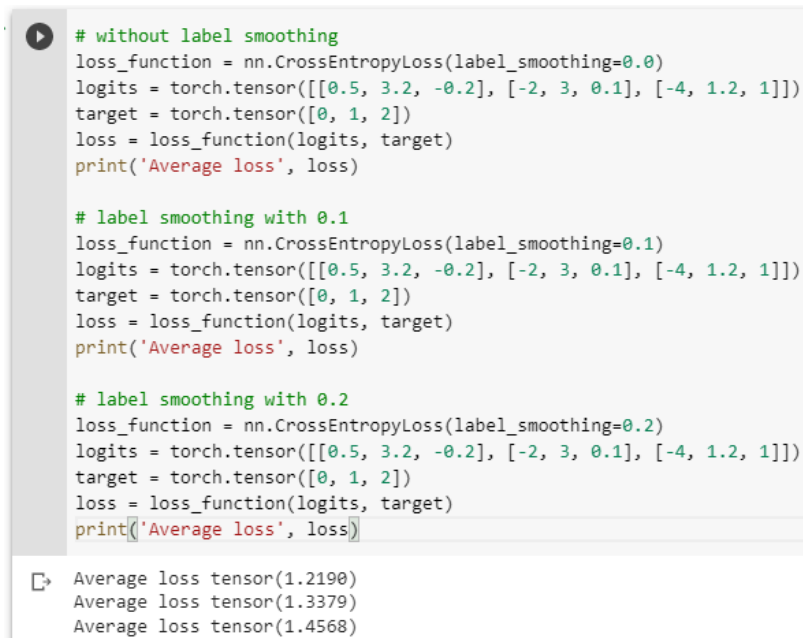
In traditional one-hot encoding, the true label is represented as a vector with all elements set to zero except for the element corresponding to the correct class, which is set to one. This encoding assumes that the true label is completely certain and has no uncertainty or noise.

In label smoothing, the true label is represented as a smoothed distribution that assigns some probability mass to both the correct and incorrect classes. The idea is to encourage the model to be less confident about its predictions, thereby reducing the risk of overfitting. Instead of optimizing for a hard target of 0 or 1 for each label, the model is trained to predict a soft target that assigns some probability mass to both the correct and incorrect labels.

The amount of probability mass assigned to the incorrect labels is controlled by a smoothing parameter, which is typically set to a small value such as 0.1 or 0.2. This small amount of probability mass ensures that the model still receives a penalty for incorrect predictions, but the penalty is not as severe as with traditional one-hot encoding.

Label smoothing is a useful technique for improving model generalization and reducing overfitting, especially in situations where the training data is noisy or contains label errors. However, it should be used with care, as too much smoothing can lead to underfitting and poor performance on the test set.

The label smoothing can be set easily as shown in Fig. 3.



```
# without label smoothing
loss_function = nn.CrossEntropyLoss(label_smoothing=0.0)
logits = torch.tensor([[0.5, 3.2, -0.2], [-2, 3, 0.1], [-4, 1.2, 1]])
target = torch.tensor([0, 1, 2])
loss = loss_function(logits, target)
print('Average loss', loss)

# label smoothing with 0.1
loss_function = nn.CrossEntropyLoss(label_smoothing=0.1)
logits = torch.tensor([[0.5, 3.2, -0.2], [-2, 3, 0.1], [-4, 1.2, 1]])
target = torch.tensor([0, 1, 2])
loss = loss_function(logits, target)
print('Average loss', loss)

# label smoothing with 0.2
loss_function = nn.CrossEntropyLoss(label_smoothing=0.2)
logits = torch.tensor([[0.5, 3.2, -0.2], [-2, 3, 0.1], [-4, 1.2, 1]])
target = torch.tensor([0, 1, 2])
loss = loss_function(logits, target)
print('Average loss', loss)
```

Average loss tensor(1.2190)
Average loss tensor(1.3379)
Average loss tensor(1.4568)

Figure 3: Higher value of label smoothing will increase the loss given the same logits prediction due to the regularization effect.

Try different values of label smoothing in the training script (by setting `label_smoothing` to any value above 0 and below 1) and see how it affects the final accuracy.

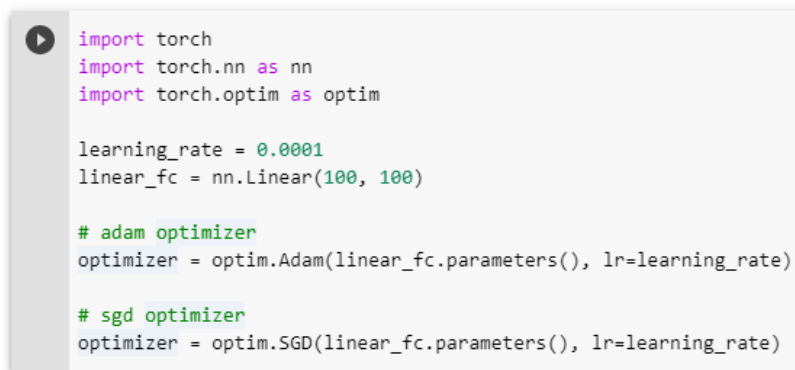
3 Optimizer

An optimizer is a key component of a machine learning algorithm that helps to adjust the parameters of a model to minimize the loss function. The goal of optimization is to find the values of the parameters that minimize the difference between the predicted output and the actual output.

The optimizer works by computing the gradients of the loss function with respect to the parameters of the model, and then updating the parameters in a way that reduces the value of the loss function. There are many types of optimizers, each with its own approach to updating the model parameters. Some popular optimizers include Stochastic Gradient Descent (SGD), Adam, Adagrad, RMSProp, and AdaDelta.

Choosing an appropriate optimizer is crucial for achieving good performance on a given task. The optimizer should strike a balance between exploring the search space to find a good solution and exploiting the current best solution. Additionally, the optimizer's learning rate, which determines how much the parameters are updated in each iteration, needs to be carefully tuned to achieve good results.

In PyTorch, you can define an optimizer as shown in Fig. 4.



```
import torch
import torch.nn as nn
import torch.optim as optim

learning_rate = 0.0001
linear_fc = nn.Linear(100, 100)

# adam optimizer
optimizer = optim.Adam(linear_fc.parameters(), lr=learning_rate)

# sgd optimizer
optimizer = optim.SGD(linear_fc.parameters(), lr=learning_rate)
```

Figure 4: Note that there are many hyperparameters in the optimizer (such as the common `weight_decay`), please read the documentation to see what hyperparameters are available.

Try to explore different types of optimizers¹ and see how they will affect classification accuracy. It can affect both the convergence speed and final accuracy. Note that you can also put `weight_decay` as a keyword argument when defining an optimizer. It is also one of the regularization techniques, try it yourself.

¹refer to <https://pytorch.org/docs/stable/optim.html#how-to-use-an-optimizer>

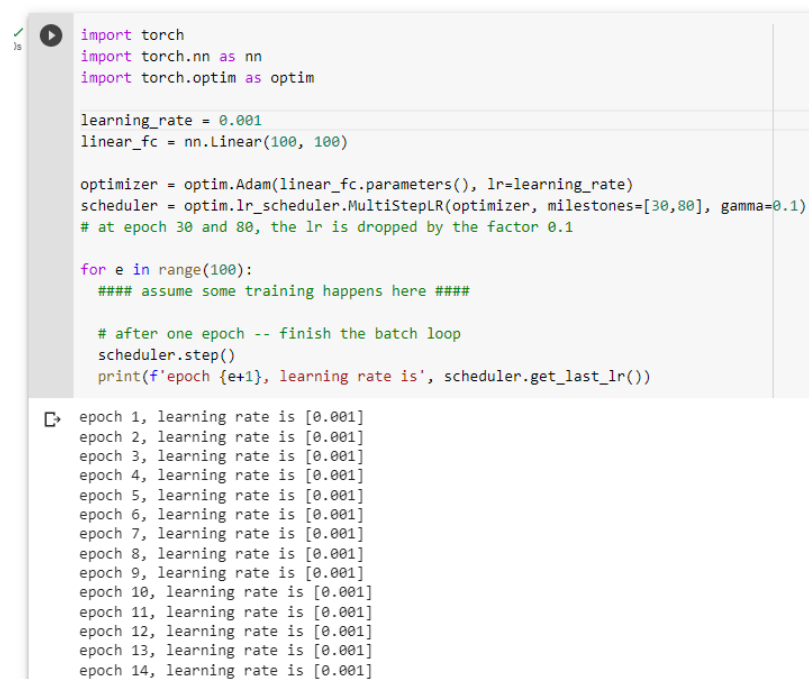
4 Scheduler

A learning rate scheduler is a component of a machine learning algorithm that adjusts the learning rate during training in order to improve the performance of the model. The learning rate determines the step size used to update the parameters of the model during optimization.

The goal of a learning rate scheduler is to improve the performance of the model by changing the learning rate at specific times or epochs during training. For example, a common approach is to start with a relatively high learning rate to quickly converge toward a solution and then gradually decrease the learning rate as the model gets closer to the optimal solution.

There are several types of learning rate schedulers², such as step decay, exponential decay, and cyclic learning rate. A step decay scheduler decreases the learning rate by a fixed factor at specific intervals during training, while an exponential decay scheduler decreases the learning rate exponentially over time. Cyclic learning rate schedules the learning rate in a cyclic manner, periodically decreasing and then increasing the learning rate to help the model explore the parameter space more effectively.

In PyTorch, you can define a multi-step learning rate scheduler as shown in Fig. 5. Note that the `.step()` function should be called at the end of one epoch in order to make the scheduling work.



```
import torch
import torch.nn as nn
import torch.optim as optim

learning_rate = 0.001
linear_fc = nn.Linear(100, 100)

optimizer = optim.Adam(linear_fc.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30,80], gamma=0.1)
# at epoch 30 and 80, the lr is dropped by the factor 0.1

for e in range(100):
    ### assume some training happens here ###

    # after one epoch -- finish the batch loop
    scheduler.step()
    print(f'epoch {e+1}, learning rate is', scheduler.get_last_lr())
```

epoch 1, learning rate is [0.001]
epoch 2, learning rate is [0.001]
epoch 3, learning rate is [0.001]
epoch 4, learning rate is [0.001]
epoch 5, learning rate is [0.001]
epoch 6, learning rate is [0.001]
epoch 7, learning rate is [0.001]
epoch 8, learning rate is [0.001]
epoch 9, learning rate is [0.001]
epoch 10, learning rate is [0.001]
epoch 11, learning rate is [0.001]
epoch 12, learning rate is [0.001]
epoch 13, learning rate is [0.001]
epoch 14, learning rate is [0.001]

Figure 5: Note that optimizer must be passed into the scheduler so that the inner part of the scheduler can adjust the learning rate.

Choosing an appropriate learning rate scheduler can help to speed up training, reduce the likelihood of overfitting, and improve the generalization performance of the model. Try different schedulers to see how they will affect classification accuracy. It can affect both the convergence speed and final accuracy.

²refer to <https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate>